



Introduzione a SQL

Python

Cos'è SQL?

SQL significa Structured Query Language ed è un linguaggio utilizzato per interagire con i database relazionali.

Permette di:

- leggere i dati presenti nel database;
- inserire nuovi dati;
- modificare o eliminare dati esistenti;
- creare e modificare tabelle;
- filtrare, ordinare e aggregare le informazioni.



Come iniziare a usare SQL

Per utilizzare SQL sul computer servono principalmente un DBMS e uno strumento con cui scrivere ed eseguire le query. Noi per iniziare possiamo utilizzare MySQL, molto diffuso e semplice da usare.

2. Installare MySQL Server dopo averlo scaricato

Il server MySQL è il componente che:

- crea e gestisce i database;
- memorizza i dati;
- riceve ed esegue le query SQL.

Durante l'installazione viene normalmente configurato anche un utente amministratore, ad esempio:

- utente: root
- password: scegliela, oppure normamamente si usa root ATTENZIONE!! se la perdi è un piccolo/grande problema

3. Installare MySQL Workbench

MySQL Workbench è l'interfaccia grafica utilizzata per collegarsi al server e scrivere query SQL.

La connessione locale utilizza normalmente:

- Host: localhost
- Porta: 3306
- User: root

4. Creare un primo database

Una volta aperto l'editor SQL:

```
CREATE DATABASE scuola;
```

Poi selezioniamo il database:

```
1. USE scuola;
```

- 5. Verificare che SQL funzioni

Possiamo eseguire una prima query:

```
1. SELECT 'Hello SQL';
```

Se viene restituito il risultato, l'ambiente è pronto.

Flusso completo

Installare MySQL

↓

Installare MySQL Workbench

↓

Connettersi a localhost

↓

Creare un database

↓

Scrivere ed eseguire SQL



SQL non è legato a un singolo database: viene utilizzato, con alcune differenze di sintassi, in sistemi come MySQL, PostgreSQL, Microsoft SQL Server, Oracle Database e SQLite.

1. *SELECT nome, cognome*
2. *FROM studenti;*

Questa istruzione richiede al database di recuperare le colonne nome e cognome dalla tabella studenti.



Database, DBMS e SQL

Un database è una struttura organizzata utilizzata per memorizzare dati in modo persistente.

Nel modello relazionale, le informazioni vengono rappresentate principalmente attraverso tabelle, composte da righe e colonne, sulle quali possono essere applicati vincoli, relazioni e regole di integrità.

Un DBMS – Database Management System è il software che gestisce fisicamente e logicamente il database, quindi, si occupa di memorizzazione, accesso concorrente ai dati, gestione delle transazioni, sicurezza, indicizzazione e controllo dell'integrità delle informazioni.



SQL rappresenta invece il linguaggio utilizzato per comunicare con il DBMS. L'applicazione invia un'istruzione SQL, il DBMS la analizza, determina come eseguirla e restituisce il risultato.

Esempi di DBMS relazionali:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database
- SQLite

1. *SELECT **

2. *FROM studenti;*

Il comando SQL richiede tutti i dati della tabella studenti, ma è il DBMS a individuare fisicamente i dati, eseguire la query e restituire il risultato.

Schema concettuale:

Applicazione



SQL



DBMS



Database



Il modello relazionale

Nel modello relazionale, i dati vengono organizzati in tabelle. Ogni tabella rappresenta una specifica entità del dominio, ad esempio Studenti, Clienti, Prodotti o Ordini.

Una tabella è composta da:

- colonne, che rappresentano gli attributi dell'entità;
- righe, che rappresentano le singole istanze memorizzate;
- vincoli, che definiscono le regole che i dati devono rispettare.



Tecnicamente, ogni riga viene spesso chiamata record o tupla, mentre ogni colonna rappresenta un attributo caratterizzato da un tipo di dato.

Il modello relazionale permette inoltre di collegare più tabelle tra loro tramite chiavi, evitando di duplicare inutilmente le informazioni.

In questo modo SQL può interrogare dati distribuiti su più tabelle mantenendo una struttura coerente e normalizzata.

id	nome	cognome	eta
1	Luca	Rossi	22
2	Anna	Bianchi	25
3	Marco	Verdi	21



Tabelle e tipi di dato

Ogni tabella SQL è definita attraverso una serie di colonne, e ogni colonna possiede un tipo di dato, di base, il tipo stabilisce quali valori possono essere memorizzati e come il DBMS deve interpretarli e gestirli.

I tipi più comuni sono:

- **INT** → numeri interi
- **VARCHAR(n)** → stringhe
- **DATE** → date
- **DECIMAL(p,s)** → numeri decimali
- **BOOLEAN** → vero/falso

```
1. CREATE TABLE studenti (  
2. id INT,  
3. nome VARCHAR(50),  
4. cognome VARCHAR(50),  
5. eta INT,  
6. data_iscrizione DATE  
7.);
```



Dal punto di vista tecnico, la scelta del tipo di dato influenza occupazione di memoria, validazione dei valori, ordinamento, confronti e performance delle query, ma soprattutto possibili condizioni:

1. *eta INT*

permette confronti numerici come:

1. *WHERE eta > 18;*

mentre:

1. *nome VARCHAR(50)*

è pensato per dati testuali.



Chiavi e relazioni tra tabelle

Nel modello relazionale, le tabelle vengono collegate tra loro tramite chiavi e questo permette di evitare duplicazioni inutili e di mantenere i dati organizzati in strutture separate ma logicamente connesse.

La Primary Key identifica in modo univoco ogni record di una tabella. Non dovrebbe contenere valori duplicati e viene spesso utilizzata con una colonna numerica come id.



```
1. CREATE TABLE studenti (  
2.   id INT PRIMARY KEY,  
3.   nome VARCHAR(50),  
4.   cognome VARCHAR(50)  
5.);
```

La Foreign Key viene invece utilizzata per creare un collegamento verso la Primary Key di un'altra tabella.

```
1. CREATE TABLE iscrizioni (  
2.   id INT PRIMARY KEY,  
3.   studente_id INT,  
4.   corso_id INT,  
5.  
6.   FOREIGN KEY (studente_id)  
7.     REFERENCES studenti(id)  
8.);
```



In questo caso studente_id contiene un riferimento a un record presente nella tabella studenti.

Dal punto di vista tecnico, le Foreign Key permettono al DBMS di applicare il principio di integrità referenziale, impedendo la creazione di riferimenti verso dati inesistenti.

Esempio concettuale:

STUDENTI		ISCRIZIONI	
id	nome	id	studente_id
---	-----	---	-----
1	Luca	1	1
2	Anna	2	2

La relazione può essere letta come:

studenti.id

↑

/

iscrizioni.studente_id



Le principali categorie di comandi SQL

SQL comprende diverse famiglie di istruzioni, ognuna pensata per operazioni differenti sul database.

Le principali sono:

- **DDL – Data Definition Language: definisce la struttura del database.**
- **DML – Data Manipulation Language: inserisce, modifica ed elimina i dati.**
- **DQL – Data Query Language: interroga e legge i dati.**
- **TCL – Transaction Control Language: gestisce le transazioni.**
- **DCL – Data Control Language: gestisce utenti e permessi.**



Le Esempi delle principali categorie sono:

- **DDL –**

- 1. *CREATE TABLE studenti (...);*

- **DML –**

- 1. *INSERT INTO studenti VALUES (...);*

- **DQL –**

- 1. *SELECT * FROM studenti;*

- **TCL –**

- 1. *COMMIT;*



Dal punto di vista tecnico, SQL non serve quindi solamente a effettuare query: permette di gestire struttura, contenuto, sicurezza e consistenza transazionale del database.

Per una prima introduzione, le istruzioni più importanti saranno:

CREATE

INSERT

SELECT

UPDATE

DELETE



Quello che poi nella programmazione chiameremo CRUD

Dalla struttura alla prima query SQL

Una volta definito il database, il passo successivo consiste nell'interrogare i dati contenuti nelle tabelle.

L'istruzione fondamentale per la lettura dei dati è:

1.SELECT

La struttura minima di una query è:

*1.SELECT colonna
2.FROM tabella;*

Esempio:

*1.SELECT nome
2.FROM studenti;*



È possibile richiedere più colonne:

1. *SELECT nome, cognome, eta*
2. *FROM studenti;*

Oppure tutte le colonne:

1. *SELECT **
2. *FROM studenti;*

Tecnicamente il DBMS riceve la query, esegue il parsing sintattico, verifica la presenza di tabelle e colonne, costruisce un piano di esecuzione e restituisce un result set.



Buon Davante a tutti

